CartLess Checkout: A Tag-Free Self-Checkout System with Smartphone Scanning, Wallet Payment and Camera-Verified One-Time Exit Passes

Hardik Hemant Rawat
Dept. of Electronics and Telecommunication
SIES Graduate School of Technology
Navi Mumbai, India
email address or ORCID

Shivraj Morde
Dept. of Electronics and Telecommunication
SIES Graduate School of Technology
Navi Mumbai, India
email address or ORCID

Ismail Mohammed
Dept. of Electronics and Telecommunication
SIES Graduate School of Technology
Navi Mumbai, India
email address or ORCID

Pranav Anand
Dept. of Electronics and Telecommunication
SIES Graduate School of Technology
Navi Mumbai, India
email address or ORCID

Swati Rane
Dept. of Electronics and Telecommunication
SIES Graduate School of Technology
Navi Mumbai, India
email address or ORCID

***Abstract*—Checkout queues remain the slowest step of shopping in small stores, and the smart carts reported in the literature either attach an RFID tag to every product, send the shopper to a cashier, or have no check at the exit. This paper presents CartLess Checkout, a self-checkout system in which shoppers scan the barcodes already printed on products with their own smartphone, pay from an in-app prepaid wallet, and leave after a Raspberry Pi camera verifies a one-time exit pass. Each pass carries a 64-bit truncated HMAC-SHA256 tag computed with a key held only by the server, and the server records its first use, so edited passes are refused and any pass, including a copy, is accepted only once. Wallet top-up tokens are signed in the same way and are credited only after staff confirm the payment. In server-side tests, all 360 forged or edited passes, all 90 reused passes and all 50 edited top-up tokens were refused, while every genuine pass was accepted on first use; the median server processing time was 9.8 ms for a scan and 10.2 ms for an exit check. A load-cell weight check was designed but is disabled in the present prototype.**

***Index Terms*—Barcodes, HMAC, Internet of Things, QR codes, Raspberry Pi, retail automation, self-checkout.**

I. Introduction

Billing at a counter is often the slowest part of a shopping trip. Customers queue even to pay for two or three items, every till needs a cashier, and the check at the exit that a customer has paid is usually done by eye against a paper bill that is easy to copy. Large retailers have begun to deploy scan-and-go carts, but such carts carry their own scanners, screens and weighing hardware, which is costly for a neighbourhood store.

Research prototypes of smart carts [1]–[5] reduce queues by billing inside the cart, but each leaves a gap. Systems based on RFID [3], [5] need a tag on every product; camera carts [1], [2] put a computer and display on every trolley and still send the shopper to a cashier to pay; and only one of the five [4] checks anything at the exit.

This paper presents CartLess Checkout, which moves scanning and payment to the shopper's own phone and concentrates the hardware at a single exit gate. Its contributions are:

1) a tag-free self-checkout flow that scans the barcodes already printed on packs, with payment from a prepaid wallet whose top-ups are confirmed by staff;
2) a one-time exit pass, signed with a truncated HMAC [6], [7], that a Raspberry Pi camera verifies with a green or red signal; and
3) a server-side evaluation of the pass and token checks and of processing time, with the prototype's limitations stated.

Section II reviews related work, Section III describes the design, Section IV the implementation, Section V the results, and Section VI concludes.

II. Related Work

Kumar et al. [1] built a Raspberry Pi 4 trolley in which a USB camera reads each item's QR code and a 20x4 LCD shows the bill; payment is made at the counter or on a phone, and no results are reported. Ibrahim et al. [2] identify products on a Raspberry Pi 4B both by barcode and with a YOLO11s detector trained on eight classes, but the receipt is paid at the cashier and scanning failed in poor light. Shahroz et al. [3] read RFID tags with an MFRC522 reader on an Arduino Uno and send the bill to an Android app over Bluetooth.

Nishitha et al. [4] scan barcodes with an Android phone, take payment by OTP or card, and place RFID detectors at the exit that sound an alarm for unbilled tags; they report that about 9, 30 and over 100 items reached the server over 2G, 3G and 4G links. Walid et al. [5] combine RFID with a load-cell weight check on a Raspberry Pi 3B+ and report 91.7% anti-theft accuracy and 93 of 100 successful RFID reads. Table I summarises the five systems.

TABLE I
COMPARISON WITH RELATED WORK

Work	Identification	Payment	Anti-theft
[1]	QR, cart camera	Counter / phone	None
[2]	Barcode + YOLO	Cashier	None
[3]	RFID tag per item	Android app	None
[4]	Barcode, phone	OTP / card	RFID exit gate
[5]	RFID tag per item	Payment link	In-cart weight
Ours	Barcode, own phone	In-app wallet	Signed exit pass

None of the five combines tag-free identification, payment inside the application and a cryptographically verified exit; CartLess Checkout is designed around that gap.

III. SYSTEM DESIGN

A. Architecture

CartLess Checkout has four parts (Fig. 1). The shopper's phone runs a web application that scans codes and shows the cart, wallet and bill. A Flask server with a MySQL database holds the catalogue, carts, orders and the wallet ledger, and is the only component that knows the signing key K. Staff use the same web application for the wallet counter, the exit gate and a sales dashboard. A Raspberry Pi 4 with a Pi Camera v2, two LEDs and a buzzer forms the exit gate. With its firewall enabled the Pi accepts no incoming connections: it calls the server over HTTPS with a device key and acts on the answer.

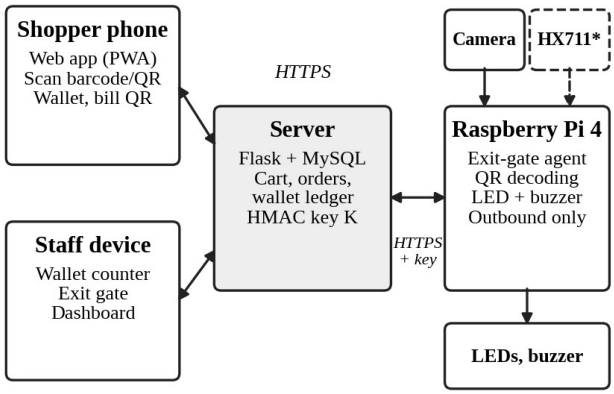


Fig. 1. System architecture. *The load cell and HX711 are built but disabled.

B. Scanning, Cart and Wallet

One camera scanner in the browser reads EAN-13, EAN-8, UPC-A, UPC-E, Code 128 and Code 39 barcodes and the store's QR labels for loose items. The server adds a product only if the code exists in the catalogue, so an unknown or mistyped code is refused; check digits are verified when staff add a product. Prices are always read from the database, never from the phone.

Bills are paid from a prepaid wallet. To add money, the shopper requests a top-up token, pays that amount at the counter in cash or by UPI, and a staff member scans the token and confirms the payment; only then is the wallet credited. Checkout locks the cart, stock and wallet rows, and an idempotency key prevents a retried request from billing twice. A bill larger than the balance is refused without charging anything.

C. Signed One-Time Exit Pass

When an order o of user u is paid, the server issues an exit pass shown as a QR code containing the text SCBILL-o-s, where

$$s = \mathrm{trunc}_{64}(\text{HMAC-SHA256}(K, m)) \quad (1)$$

where m = "smartcart-bill:" ‖ o ‖ ":" ‖ u, K is the server's secret key, and trunc_{64} keeps the first 64 bits [6], [7]. Without K, an attacker who edits the order number or the tag in (1) must guess a 64-bit value, so a single guess succeeds with probability

$$P_{\text{forge}} = 2^{-64} \approx 5.4 \times 10^{-20}. \quad (2)$$

Signing alone cannot stop a copy of a genuine pass, such as a screenshot. The server therefore records the first exit for each order; any later presentation is refused as already used. Top-up tokens SCTOP-t-s are signed in the same way over the token and user identifiers, and a token is credited exactly once.

D. Exit Verification

At the gate the shopper holds the pass under the Pi camera. The Pi decodes the QR code and sends it to the server, which applies the checks of Fig. 2 in order. The Pi shows the answer: a green LED and one beep for a valid pass; a red LED with a long beep for an invalid one and two beeps for a pass already used. A pass is also held, without being consumed, while the shopper has unresolved flagged scans. Staff can verify the same pass with a phone, and every Pi result appears live on the staff exit-gate page.

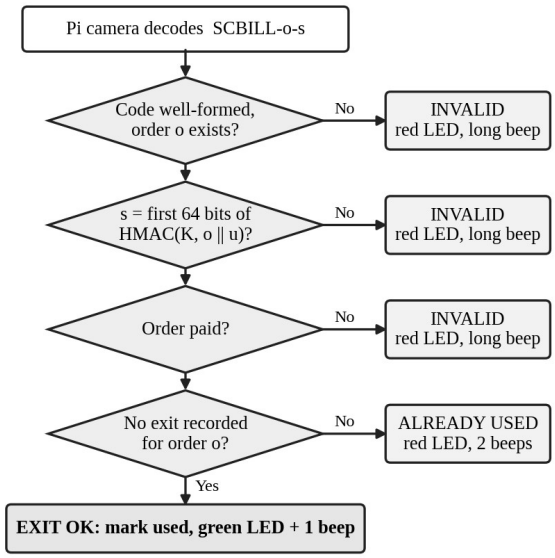


Fig. 2. Exit-pass verification performed by the server for each code read by the Pi.

E. Weight Verification

A weight check was designed to catch an item placed in the cart without being scanned or swapped for another. A 10 kg

load cell with an HX711 amplifier reports one weight per placement; an item is accepted only if its weight lies within ±3% of the product's calibrated value and, optionally, its camera colour matches a reference. Each reading may verify only one scan. In the present prototype the HX711 board failed during testing, so this check is disabled and the evaluation in Section V does not include it.

IV. Implementation

A. Hardware

Table II lists the exit-gate hardware and its connections to the Raspberry Pi 4 header. The LEDs are driven through 220 Ω resistors and the camera uses the CSI port.

TABLE II
HARDWARE AND PIN CONNECTIONS

Part	Function	Pi pin (GPIO)
Raspberry Pi 4 Model B	Exit-gate controller	–
Pi Camera v2 (8 MP)	QR decoding	CSI port
Green LED	Pass accepted	11 (GPIO 17)
Red LED	Pass refused	13 (GPIO 27)
Active buzzer	Audible result	15 (GPIO 22)
HX711 DT / SCK*	Load-cell ADC	29 / 31 (GPIO 5 / 6)

B. Software

The server uses Python with Flask and SQLAlchemy over MySQL 8 and is hosted on Render with the database on Aiven. The website is plain HTML, CSS and JavaScript, installable as a progressive web app, and bundles its scanner library so that it works on networks that block public content-delivery networks. The Pi agent is a Python program using Picamera2 for capture, pyzbar for QR decoding and lgpio for the LEDs and buzzer; it starts at power-up and sends a heartbeat every 30 s.

Further controls protect the system: passwords are stored as hashes and five failed logins lock an account for 15 minutes; requests are rate-limited per logged-in user, or per address when anonymous; a Content-Security-Policy permits only the site's own scripts and Google sign-in; and all keys come from the server's configuration rather than the code. Figs. 3 and 4 show the shopper's screens and the staff wallet counter running on the Pi's own display.

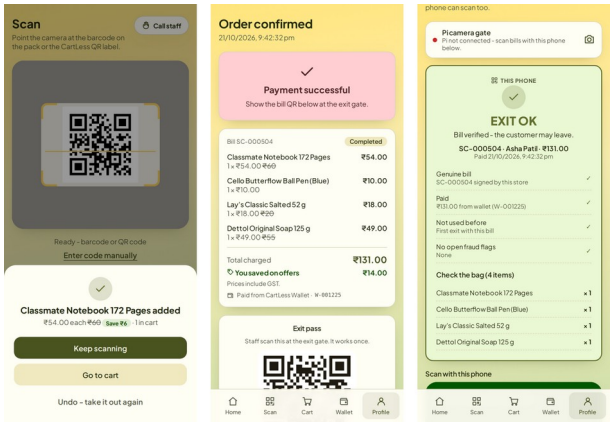


Fig. 3. Shopper screens: scan, paid bill with exit pass, and exit-gate result.

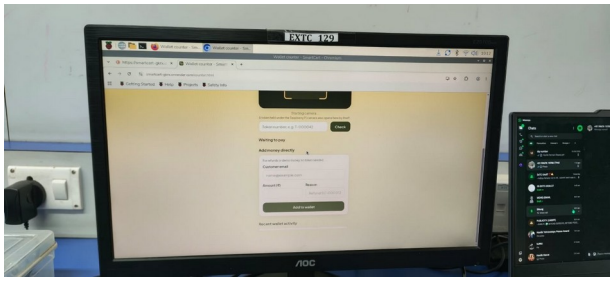


Fig. 4. CartLess wallet counter running on the Raspberry Pi display in the laboratory.

V. Results and Discussion

A. Test Setup

The checks were exercised through the server's own interface using the Flask test client against MySQL 8.0 on a test machine with two cores of a 2.1 GHz Intel Xeon processor, with rate limiting disabled. The test created shoppers, scanned items, credited each wallet with ₹2,000 directly by staff and checked out 100 carts: 90 were paid and 10 whose total exceeded the balance were correctly refused. Exit passes were submitted through the endpoint the Pi uses, with its device key. Times are server processing times only; network transfer, QR decoding on the Pi and the camera are not included.

B. Security Tests

Table III summarises the results. For each of the 90 paid orders the genuine pass was presented twice, and four counterfeit passes were made: one with a single hexadecimal digit of the tag changed, one carrying the tag of a different paid order, one with a random 64-bit tag, and one naming an order that does not exist.

TABLE III
PASS AND TOKEN VERIFICATION RESULTS

Test	Expected	Outcome
Genuine pass, first use	Accept	90/90 accepted
Same pass again	Refuse (used)	90/90 refused
One tag digit changed	Refuse (invalid)	90/90 refused
Tag of another paid order	Refuse (invalid)	90/90 refused
Non-existent order number	Refuse (invalid)	90/90 refused
Random 64-bit tag	Refuse (invalid)	90/90 refused
Edited top-up token	Refuse	50/50 refused
Top-up confirmed twice	Credit once	50/50 credited once
Unknown or mistyped code	Refuse	300/300 refused
Missing or wrong device key	Refuse	HTTP 401

Every genuine pass was accepted once and refused thereafter, and no counterfeit pass or token was accepted. The result for counterfeits is expected from (2); the tests confirm that the implementation checks the tag, the order and the used state on every request.

C. Processing Time

Fig. 5 shows the distribution of server processing time. The median was 9.8 ms (95th percentile 13.3 ms) for a scan, 19.1 ms (25.1 ms) for a wallet checkout, and 10.2 ms (12.9 ms) for an exit-pass check through the Pi's endpoint. Unknown codes were refused in a median of 2.4 ms. These times are small

compared with the network delay of a mobile connection, so the shopper's wait is expected to be set by the network and the camera rather than by the server.

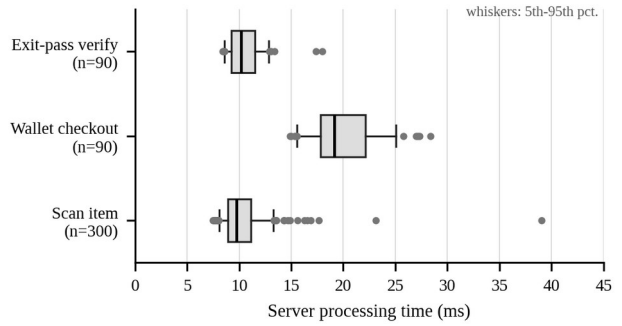


Fig. 5. Server processing time per request (boxes: quartiles; whiskers: 5th–95th percentile; points: outliers).

D. Limitations

Three limitations remain. First, with the weight check disabled, the exit pass proves that a bill was paid but not that the bag matches it; staff still compare the bag with the item list shown at the gate. Second, the pass is a bearer token: a copy presented before the genuine pass is accepted once, after which the owner is refused and the gate shows when and by whom the bill was used. Third, the evaluation is server-side; end-to-end timing on the Pi, QR decoding accuracy under store lighting, and a trial with shoppers have not yet been carried out.

VI. Conclusion and Future Work

The CartLess Checkout prototype suggests that a small store can offer self-checkout without tagging products or fitting a computer to every cart: shoppers scan existing barcodes with their own phones, pay from a staff-confirmed wallet, and leave after a Raspberry Pi verifies a signed, one-time exit pass. In testing, every forged, edited or reused pass and token was refused at a median server cost of about 10 ms. Future work will restore the weight check with a working HX711 and per-product tolerances, weigh the bag at the exit, add a UPI payment gateway, recognise products with a small on-device detector as in [2], and measure the complete system in a store trial.

Acknowledgment

The authors thank Prof. Pratibha Joshi, Mini Project Coordinator, and the Department of Electronics and Telecommunication, SIES Graduate School of Technology, for laboratory facilities and the Raspberry Pi used in this work.

References

[1] M. Kumar et al., “Intelligent shopping trolley with automated billing using Raspberry Pi,” in Proc. 4th Int. Conf. Smart Technologies in Computing, Electrical and Electronics (ICSTCEE), Bengaluru, India, 2023, doi: 10.1109/ICSTCEE60504.2023.10585242.

[2] N. Ibrahim et al., “Smart shopping cart: A Raspberry Pi-based system for reducing supermarket checkout delays,” in Proc. IEEE Int. Conf. Smart Sustainable Systems for Computer and Engineering Applications (3SCEA), 2026, pp. 1100–1109, doi: 10.1109/3SCEA68071.2026.11602934.

[3] M. Shahroz, M. F. Mushtaq, M. Ahmad, S. Ullah, A. Mehmood, and G. S. Choi, “IoT-based smart shopping cart using radio frequency identification,” IEEE Access, vol. 8, pp. 68426–68438, 2020, doi: 10.1109/ACCESS.2020.2986681.

[4] R. Nishitha, S. S. Naik, V. Raksha, and U. Kulsum, “IoT based automatic billing system using barcode scanner by Android device and monitoring unregistered barcode by RFID,” in Proc. 4th Int. Conf. Recent Trends on Electronics, Information, Communication & Technology (RTEICT), Bangalore, India, 2019, pp. 15–20, doi: 10.1109/RTEICT46194.2019.9016698.

[5] M. H. Walid et al., “The smart cart with automatic billing and anti-theft,” in Proc. IEEE Int. Conf. Electrical, Electronic and Computer Engineering (ICEECE), 2026, doi: 10.1109/ICEECE71300.2026.11709931.

[6] H. Krawczyk, M. Bellare, and R. Canetti, “HMAC: Keyed-hashing for message authentication,” Internet Engineering Task Force, RFC 2104, Feb. 1997. [Online]. Available: https://www.rfc-editor.org/rfc/rfc2104

[7] National Institute of Standards and Technology, “The keyed-hash message authentication code (HMAC),” FIPS PUB 198-1, Jul. 2008. [Online]. Available: https://doi.org/10.6028/NIST.FIPS.198-1